

# OS Embarqués pour l'Edge Computing

## Conception bout en bout d'un *smart sensor/actuator* avec gateway optimisée

⚠ Document sujet à évolution dans les prochaines semaines

### Introduction

Ce projet est un travail d'intégration visant à mettre en pratique les concepts de programmation de contrôleurs sans système d'exploitation (*bare-metal*), de systèmes d'exploitation temps réel (RTOS), et de communication inter-périphériques, dans le contexte des Systèmes Cyber-Physiques (CPS) et de l'Internet des Objets (IoT).

L'objectif principal est de concevoir, d'implémenter et de valider une architecture complète, allant du capteur/actionneur physique jusqu'à une passerelle de niveau *fog/edge*, en justifiant tous les choix de conception.

Le rendu est attendu par **courriel** à Julien Deantoni : `firstname.lastname@univ-cotedazur.fr`.

- **Objet du courriel** : Doit commencer par [IoT-CPS-EdgeComp] suivi de projet équipe X, où X est le nom de votre équipe (tel qu'utilisé sur le canal Slack dédié).
- **Date Limite** : Le rendu est attendu **avant le 18 Janvier 2026 à 22h00** (Heure de Paris).
- **Format** : Le rendu doit être un **document PDF** au format d'un article scientifique classique (court mais pertinent, **maximum 10 pages**).

### Rendu écrit

Le document doit impérativement contenir les éléments suivants, structurés comme les sections d'un article scientifique :

- Les noms des membres de votre équipe.
- Un **lien vers le code** de votre système (typiquement un lien vers le dépôt Git. Ce dépôt doit clairement expliquer comment configurer et utiliser le projet).
- Une section **Introduction** spécifiant le contexte d'utilisation du système, les exigences fonctionnelles et extra-fonctionnelles que vous avez jugées les plus importantes (avec une explication de ce choix, à détailler dans la section "Solution Proposée").
- Une section **Solution Proposée** spécifiant l'architecture matérielle, l'architecture applicative, et la spécification de déploiement. Cette section doit présenter une **analyse critique** de votre propre solution, détaillant ce qui a bien fonctionné et ce qui pourrait être amélioré (et comment).
- Une section **Implémentation et Résultats** mettant en lumière comment et pourquoi votre solution est (ou non) une bonne solution.
- Une **Conclusion** résumant les principaux avantages et inconvénients de votre architecture, la répartition des responsabilités de chaque membre de l'équipe, ainsi que les perspectives d'évolution potentielles du projet.

## Présentation Orale (Semaine 8)

Chaque groupe réalisera une présentation formelle de son projet. La présentation devra :

- Synthétiser l'approche technique.
- Démontrer le fonctionnement du système complet (du capteur à la passerelle).
- Discuter les défis rencontrés et la manière dont ils ont été résolus.
- Mettre en évidence les avantages liés à l'utilisation du RTOS et à l'optimisation de la passerelle.

## 1 Domaine Applicatif : Le Capteur/Actionneur Intelligent (Smart Sensor/Actuator)

Chaque groupe est invité à choisir un domaine applicatif en rapport (ou non) avec ses autres projets dans la mineure, en utilisant le matériel disponible parmi une liste pré-établie de matériel disponible (voir Section 3). Le projet consiste à développer un système complet basé sur l'architecture suivante :

1. **Smart Sensor/Actuator (Arduino)** : Un microcontrôleur agissant comme un "Smart Sensor/Actuator" capable d'exécuter des tâches temps réel (acquisition, traitement local, contrôle d'actionneur). Il doit fonctionner de manière autonome.
2. **Passerelle Edge (Raspberry Pi)** : Un système hôte utilisant un protocole de communication classique (I2C) pour interagir avec l'Arduino, collecter ses données traitées, lui envoyer des commandes, et servir de point de jonction avec le réseau (Internet/Cloud).

Le choix du cas d'usage (ex.: station météo temps réel, contrôleur d'accès, etc.) est libre, mais doit impérativement être validé par l'équipe enseignante en semaine 1.

## 2 Progression Technique du Projet (sur 7 Semaines)

La progression est structurée en trois phases techniques obligatoires, chacune exigeant une montée en complexité progressive et l'intégration de différents périphériques et d'optimisation.

### 2.1 Phase 1: Prise en Main de la Programmation $\mu$ -Contrôleur (1 séance)

Vous allez commencer par prendre en main la programmation micro contrôleur basique en faisant les exercices suivants:

- prise en main de l'environnement de développement. Pour ceci, télécharger les ressources de la séance 1 et prenez en main l'exemple "led blink".
- Vous devez ensuite réaliser un système simple dont la spécification est par exemple, allumer la led lors d'un appui sur un bouton et l'éteindre lors de l'appui suivant.
- Vous continuerez ensuite en mettant en place des interruptions. Par exemple l'appui sur un bouton déclenchera une interruption dont le *handler* se chargera de changer une variable globale<sup>1</sup>. **Lisez l'introduction du chapitre 13 des datasheets.** Vous pouvez utiliser l'interruption PCINT2 (Pin Change Interrupt 2) déclenché par un changement d'état de PCINT20. Positionner le bouton poussoir en conséquence. Il s'agit ensuite de paramétrer les bons registres du contrôleur afin d'autoriser l'interruption PCINT2. Ceux-ci sont décrits dans le chapitre 13 des datasheets.

<sup>1</sup>vous avez lu variable globale et vos cheveux se sont hérissés ? Il n'y a pas de quoi. C'est la manière classique de communiquer entre handler et code principale en programmation micro-contrôleur. Cependant vous avez eu raison de réagir car une personne attentive ira lire cette document claire et utile: [https://www.nongnu.org/avr-libc/user-manual/group\\_util\\_atomic.html](https://www.nongnu.org/avr-libc/user-manual/group_util_atomic.html)

Remarque: dans le handler de l'interruption, on respecte habituellement les règles suivantes:

- on reste le minimum de temps possible !
- Souvent, on commence par désactiver l'interruption qui nous a déclenché, on fait le traitement, puis on réactive l'interruption. Le “souvent” signifie lorsque que l'on ne veut pas traiter des rafales d'interruptions TRÈS rapprochées dans le temps. Voir: “cela dépend de la sémantique de l'interruption” dans le cours.
- pour des raisons de rebonds électrique lors de l'appui sur le bouton poussoir, vous mettrez un petit délai dans le handler; cette fonctionnalité est habituellement nommée *debounce*. Remarquez que ce délai est en contradiction avec le premier point. Donc lorsque c'est possible on préfère éliminer ces rebonds électroniquement (<http://www.labbookpages.co.uk/electronics/debounce.html>).

- Enfin, vous utiliserez un timer avec interruption. Par exemple pour modifier l'état d'un led avec une interruption initiée par le Timer 16bit numéro 1 (pages 115 à 145 des datasheets et plus particulièrement de 136 à 142). Pour ceci sachez qu'il vous est possible de générer une interruption lorsqu'un timer atteint une valeur prédéfinie (le vecteur d'interruption associé est nommé `TIMER1_COMPA_vect`). Cette valeur doit alors être mise dans un(des) registre(s) spécifiques (`OCR1A` dans notre cas). Pour calculer cette valeur de comparaison pour le timer 1, vous devez d'abord choisir la vitesse à laquelle le timer va évoluer (sa période). Pour cela un registre spécifique permet de choisir si la période du timer est celle de l'oscillator du circuit (qui est de 1/16MHz dans notre cas) ou à un multiple de cette période (table 16-5 des datasheets). Une fois la période du timer choisie, on sait que pour une division de la vitesse de 1024 le timer s'incrémentera tous les :

$$(1024/16MHz) = 64\mu s$$

Ainsi si vous comparez votre timer à une valeur de 15625 (soit 0x3D09 en hexadecimal), vous voulez attendre 1 seconde. Attention, par défaut le timer ne revient pas à zéro une fois qu'il a atteint la valeur de `OCR1A`; il faut donc être sur de se mettre dans le mode CTC (Clear Timer on Compare match) basé sur `OCR1A`.

## 2.2 Phase 2 : Du Bare-Metal au Temps Réel (1 séance)

- **Drivers** : Sélectionner les capteurs et/ou actionneurs cruciaux pour le projet. Implémenter chacun des *drivers* de ces périphériques sur l'Arduino en vous basant sur les méthodes apprises la semaine #1.
- **Ajout de l'OS temps réel** : Migrer l'application vers **FreeRTOS**. Les tâches critiques (acquisition, contrôle) devront être implémentées comme des tâches FreeRTOS avec une gestion justifiée des priorités, des sémaphores et des variables partagées pour la communication inter-tâches.

## 2.3 Phase 2 : Communication I2C (2 séances)

- **Arduino : Maillon Esclave** : L'Arduino doit être configuré comme un périphérique **Esclave (Slave) I2C**. Définir un protocole d'échange de données clair (structure des registres, commandes, etc.) pour interroger le capteur et/ou envoyer des commandes à l'actionneur. L'Arduino doit gérer l'interrogation par le Maître de manière non-bloquante au sein de son architecture FreeRTOS.
- **Raspberry Pi : Maillon Maître** : Le Raspberry Pi (RPi) doit être implémenté comme **Maître (Master) I2C** capable de lire les données et d'envoyer les commandes au Smart Sensor/Actuator. Le RPi ne doit utiliser cette interface que comme un bus d'interrogation de périphérique.

## 2.4 Phase 3 : Passerelle Edge Optimisée (2 séances)

- **Architecture du RPi** : Développer l'application sur le Raspberry Pi. L'accent doit être mis sur l'optimisation des ressources et la réactivité. Les étudiants devront implémenter une approche d'“OS dédié” ou d'architecture applicative optimisée, par exemple en utilisant des conteneurs légers ou en se basant sur des services systèmes (démons) minimaux pour minimiser l'empreinte logicielle de la passerelle.
- **Fonctionnalité de Passerelle (Fog)** : Le RPi doit servir de passerelle, traduisant et filtrant les données I2C brutes ou formatées de l'Arduino vers un protocole réseau de plus haut niveau (ex. : MQTT, HTTP) pour l'envoi vers un service distant (simulé ou réel).

## 2.5 Phase 4 : Finalisation (1 séance)

Cette séance vous permet de finaliser le projet et les documents demandés.

# 3 Matériel à Disposition

Les équipes devront baser leur projet sur le matériel suivant :

- **Microcontrôleur** : Cartes Arduino.
- **Passerelle** : Raspberry Pi 3 ou 4.
- **Capteurs/Actuateurs** disponibles ici: [https://webusers.i3s.unice.fr/~deantoni/teaching\\_resources/SI5/SA\\_edgeComp/sensors\\_actuators\\_ubiquarium.html](https://webusers.i3s.unice.fr/~deantoni/teaching_resources/SI5/SA_edgeComp/sensors_actuators_ubiquarium.html)